Guided Portfolio Watch — Design Process

Problem and Target Persona

Most financial dashboards assume the reader already understands finance. The persona I designed for is a first-time long-term investor: someone who bought a few equities, intends to hold 3–5 years, and has no financial background. This user has no reliable way to distinguish signal from noise — or to know whether a headline actually matters for the specific businesses they own.

The problem worth solving: *show only what materially affects a long-term business case, in plain English, without requiring prior financial knowledge.*

User Insight and Product Hypothesis

The core insight came from real-user testing. A prototype displaying accurate financial metrics — "revenue up 65%, net income up 64%" — failed to communicate anything meaningful to a non-expert tester. The numbers were correct; the context was absent.

This surfaced the hypothesis: **the right unit of information is not a metric; it is an explanation.** Every alert and insight card leads with *what happened*, follows with *why it matters*, and closes with *what to watch next* — with sources. In guided mode, metric labels read "Total sales," "Profit after costs," "Cash generated."

Major Design Decisions

No price data. A beginner monitoring prices daily is likely to over-react and undermine a long-term strategy. The Skill covers material business-level changes only: earnings, regulatory decisions, strategic announcements, supply-chain shifts.

Guidance inferred, never asked. Asking "are you a beginner?" is a friction point and an identity question. Guidance mode is inferred from natural-language signals ("explain it to me" → guided; correct use of EBITDA/FCF → standard), and is overridable via AGENTS.md.

Evidence gate on alerts. After the first production run, the selected finding had no traceable external source URL. Delivering an alert with no verifiable citation would undermine user trust. The decision: suppress findings without at least one external HTTPS URL, log the reason, and treat source URL coverage as an open product question.

Single config as source of truth. Both feeds read from one `config.json` at runtime. Changing tickers produces a fully updated Playbook and alert pipeline on the next run — no code changes required.

Skill Workflow and Testing

Setup is a 7-step agent workflow. `parseIntent()` maps natural-language input deterministically to a `MonitoringConfig` — no cloud calls until the user confirms. The monitoring script wraps `@alva/portfolio-watch` with two deterministic post-LLM validation layers before any delivery.

348 local tests cover intent parsing, YoY financial logic, Playbook rendering, alternate-ticker reusability, and alert pipeline logic. 93 monitor-alert tests were added after a production run surfaced data-contract bugs and the source URL gap. One production run completed (110 events, 1 finding selected); alert generation failed due to a post-processing bug, was fixed, and redeployed. Financial-event notification has not yet been delivered end-to-end.